Name: Yash Singh	Subject: Agile Software Development & DevOps
T.E. Batch: T22 Sem: 5th Roll No: 2401118	Date: 17/10/26

Name: Yash Singh

Roll No. 2401118

Class: T22

EXPERIMENT NO. 9

Aim: To understand and implement cloud infrastructure provisioning using Terraform, an Infrastructure as Code (IaC) tool.

Theory

Terraform is an open-source Infrastructure as Code (IaC) tool developed by HashiCorp. It allows users to define, configure, and manage cloud infrastructure using human-readable configuration files. Instead of manually creating resources through a cloud provider's dashboard, Terraform automates the process using code.

Terraform supports multiple cloud providers, including Amazon Web Services (AWS), Microsoft Azure, and Google Cloud Platform (GCP). It can be used to provision resources such as virtual machines, storage buckets, networks, databases, and security groups.

Key Components of Terraform

1. Providers: Providers are plugins that allow Terraform to interact with cloud platforms and other services. For example, the AWS provider enables Terraform to create and manage AWS resources.

2. Resources: Resources represent infrastructure components that Terraform manages, such as EC2 instances, S3 buckets, and virtual networks.

3. Configuration Files: Terraform uses files with the .tf extension to define the desired infrastructure using HashiCorp Configuration Language (HCL).

4. State File: Terraform maintains a state file named terraform.tfstate to track the infrastructure it manages and compare the current state with the desired configuration.
5. Terraform CLI: The command-line interface allows users to initialize configurations, preview changes, create resources, and destroy infrastructure.

Working of Terraform

Terraform follows these steps to provision cloud infrastructure:

1. Write Configuration: The user creates a .tf file describing the required cloud resources.
2. Initialize: The terraform init command downloads the required provider plugins and initializes the working directory.
3. Validate: The terraform validate command checks the configuration for syntax and consistency errors.
4. Plan: The terraform plan command previews the infrastructure changes Terraform intends to make.
5. Apply: The terraform apply command provisions the specified resources on the cloud platform after confirmation.
6. Manage Infrastructure: Terraform tracks the provisioned resources through its state file and can update or remove them when required.

Example Terraform Configuration

The following example provisions an Amazon EC2 instance using Terraform.

```
provider "aws" {
  region = "ap-south-1"
}

resource "aws_instance" "example" {
  ami           = "ami-xxxxxxxx"
  instance_type = "t2.micro"
```

```
  tags = {
    Name = "Terraform-Instance"
  }
}
```

Note: Replace ami-xxxxxxxx with a valid AMI ID available in the selected AWS region. AWS credentials must be configured before provisioning.

Important Terraform Commands

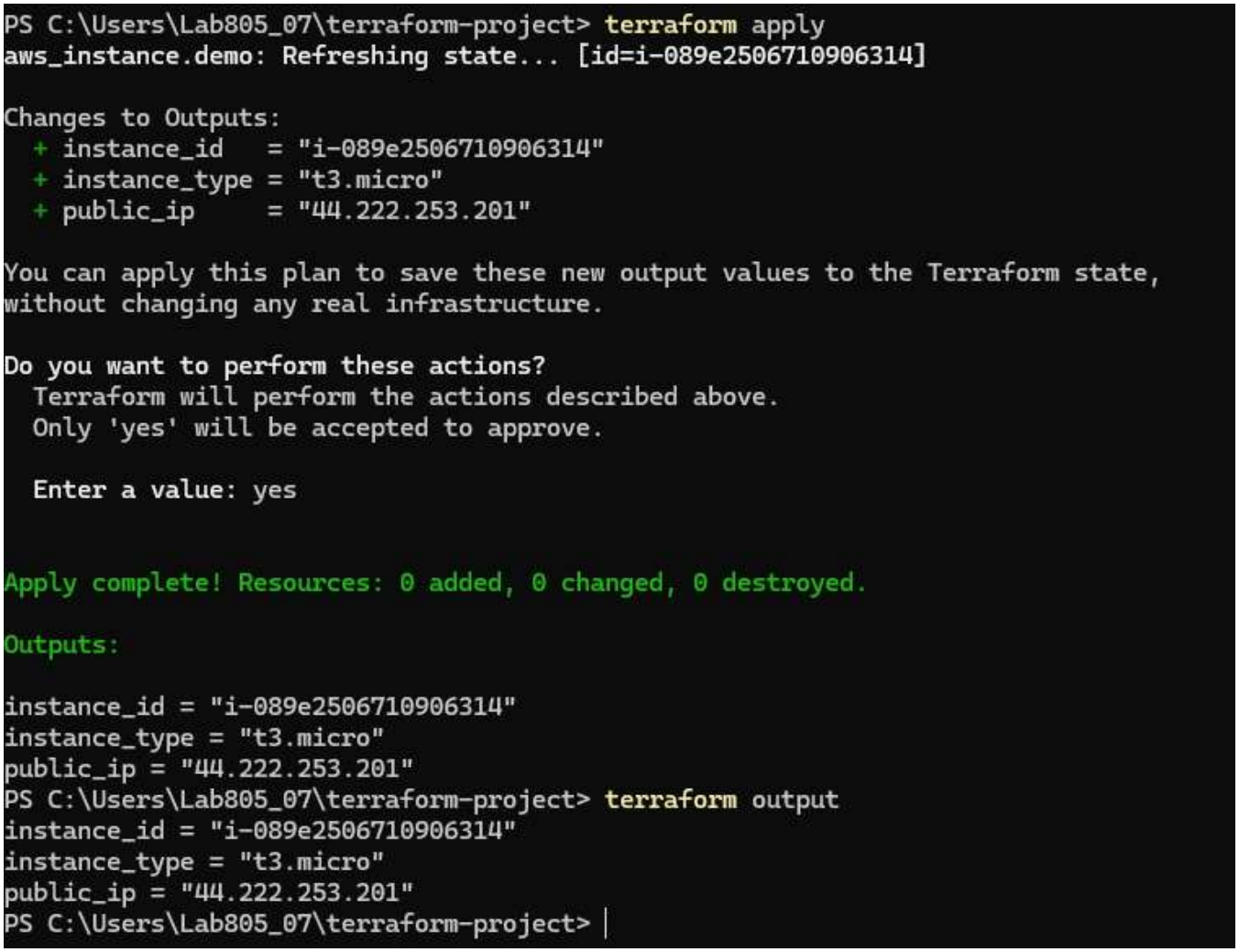
Command	Purpose
terraform init	Initializes the Terraform project.
terraform validate	Validates the configuration files.
terraform plan	Previews the proposed changes.
terraform apply	Creates or updates infrastructure.
terraform destroy	Removes resources managed by Terraform.

Advantages of Terraform

- Automation: Reduces manual effort in infrastructure setup.
- Consistency: Creates infrastructure using repeatable configurations.
- Version Control: Configuration files can be stored in Git.
- Multi-Cloud Support: Supports several cloud platforms.
- Scalability: Makes it easier to manage large infrastructure environments.

- Reusability: Modules allow configurations to be reused across projects.

Screenshot:



Conclusion

Terraform simplifies cloud infrastructure provisioning through the Infrastructure as Code approach. It enables users to define, deploy, modify, and manage cloud resources using configuration files and commands. It improves consistency, automation, scalability, and reliability while reducing the complexity of managing cloud infrastructure manually.